

Stripe Checkout & Webhook Flow — Full Walkthrough

Grund Shop: Event destinations, the success page, and the webhook handler

1. Creating an Event Destination (Webhook) in the Dashboard

In Workbench, the Webhooks tab is where you register a URL for Stripe to notify when something happens (a payment succeeds, fails, etc). Steps to create one:

1. Go to the Webhooks tab in Workbench (dashboard.stripe.com/webhooks).
2. Click "Add destination" (or "Create an event destination").
3. Event destination scope: choose "Your account" — "Connected accounts" is only for Stripe Connect marketplaces/platforms, which this project doesn't use.
4. API version: leave the default unless your code targets an older version specifically.
5. Events: do NOT select "All events." Expand Payment Intent and select only what the checkout flow needs: `payment_intent.succeeded` and `payment_intent.payment_failed`.
6. Destination type: choose Webhook (sends to an HTTPS endpoint), not AWS EventBridge or Azure Event Grid.
7. Endpoint URL: the publicly reachable HTTPS URL Stripe should POST to, e.g. `https://yourdomain.com/api/webhooks/stripe`.
8. Destination name: a purely internal label for your own reference — Stripe doesn't use it functionally. e.g. "Grund Shop Sandbox - Order Payments."
9. Create the destination. Stripe reveals a signing secret starting with `whsec_`. Copy it into your env vars (e.g. `STRIPE_WEBHOOK_SECRET`) as a Secret, not Config.

Testing locally: Stripe can't reach localhost directly (no public route to your machine). Use the Stripe CLI's local listener instead:

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

This forwards real/triggered test events to your local server and prints a temporary `whsec_` secret to use in `.env.local` while developing. Switch to the real production destination and its `whsec_` only once deployed.

2. Where `pk_` and `sk_` Are Actually Used

Key	Used where	Used for
<code>pk_</code> (publishable)	Browser only (<code>StripePayment.tsx</code>)	<code>loadStripe(NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY)</code> creates the <code>Stripe.js</code> instance, which is then passed into <code><Elements stripe={...}></code> , which renders the card form.
<code>sk_</code> (secret)	Server only	Creating the <code>PaymentIntent</code> (<code>OrderDetailsPage</code>), retrieving/verifying it (<code>Successpage</code>), and constructing/verifying webhook events.

`pk_` is used exactly once in this codebase: inside `loadStripe` on the checkout page. It never touches order creation, payment verification, or the webhook handler — those are all `sk_`.

3. The Success Page — What It Actually Does

Analogy: the checkout page is the card machine; the success page is the back-office receipt printer. It never talks to the customer's card — it double-checks the sale really happened before printing anything.

```
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY as string);

const Successpage = async (props: {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ payment_intent: string }>;
}) => {
  const { id } = await props.params;
  const { payment_intent: paymentIntentId } = await props.searchParams;

  const order = await getOrderById(id);
  if (!order) notFound();

  const paymentIntent = await stripe.paymentIntents.retrieve(paymentIntentId);

  if (paymentIntent.metadata.orderId === null ||
    paymentIntent.metadata.orderId !== order.id) {
    return notFound();
  }

  const isSuccess = paymentIntent.status === 'succeeded';
  if (!isSuccess) return redirect(`/order/${id}`);

  // render "Thanks for your purchase"
};
```

Step by step:

- When `confirmPayment` succeeds on the checkout page, Stripe redirects the browser to your `return_url` with query params appended:
`?payment_intent=pi_abc...&payment_intent_client_secret=...&redirect_status=succeeded`.
- `searchParams` here just parses those URL query params — no Stripe API call yet.
- `stripe.paymentIntents.retrieve(paymentIntentId)` is the real, authoritative check. It uses `sk_` to ask Stripe's API directly for the `PaymentIntent`'s current status and metadata — it does NOT trust anything from the URL.
- The `metadata.orderId` check prevents someone pasting a valid `PaymentIntent` ID for a different order into this URL to fake success on this one.
- The `status === 'succeeded'` check is the actual proof of payment. `redirect_status` in the URL is not proof — a user could edit that query param by hand. That's exactly why the code re-checks with Stripe instead of trusting it.

4. The Webhook — What It Is and When It Runs

Analogy: the success page is the cashier telling the customer “looks like that went through!” The webhook is the bank separately sending a confirmation slip to the back office — it doesn't care whether the customer is still there, whether their browser crashed, or whether the redirect ever completed.

The webhook runs completely independently of the browser. The moment the subscribed event happens on Stripe's servers, Stripe makes a server-to-server HTTP POST directly to your registered endpoint (e.g.

/api/webhooks/stripe). This can happen before, during, or even if the customer's browser never makes it back to the success page at all (closed tab, dropped connection, redirect failure).

Why it's needed even though the success page also checks the `PaymentIntent`: the success page's check only runs if the browser gets back to your site. If it doesn't, the payment can still have succeeded, but nothing on your side marks the order as paid — unless the webhook (a separate, guaranteed notification) does it.

Requires a public HTTPS URL: Stripe's servers need a real, internet-reachable address to POST to. localhost has no public route, so for local development use the Stripe CLI's local listener (`stripe listen --forward-to localhost:3000/...`) or a tunneling tool like ngrok. This is not a “dev vs prod” limitation — it's specifically about public reachability.

5. The Webhook Handler — Review

```
export async function POST(req: NextRequest) {
  const event = await Stripe.webhooks.constructEvent(
    await req.text(),
    req.headers.get('stripe-signature') as string,
    process.env.STRIPE_WEBHOOK_SECRET as string
  );

  if (event.type === 'charge.succeeded') {
    const { object } = event.data;
    await updateOrderToPaid({
      orderId: object.metadata.orderId,
      paymentResult: {
        id: object.id,
        status: 'COMPLETED',
        email_address: object.billing_details.email!,
        pricePaid: (object.amount / 100).toFixed(),
        name: '',
      },
    });
    return NextResponse.json({ message: 'UpdateOrderToPaid was successful' });
  }

  return NextResponse.json({ message: 'event is not charged.succeeded' });
}
```

Issues found:

- **Event type mismatch (important):** this checks `event.type === 'charge.succeeded'`, but the dashboard was configured to send `payment_intent.succeeded` / `payment_intent.payment_failed`. If the destination only sends `payment_intent.*` events, this if-block never runs, and real payments silently fall into the else branch. The event type checked in code must match what's actually selected in the dashboard.
- **No try/catch around `constructEvent`:** an invalid/missing signature throws, which currently results in a generic 500. Stripe expects a proper 400 response for bad signatures.
- **`(object.amount / 100).toFixed()` rounds to a whole number** with no decimal places — 1999 cents becomes "20", not "19.99". Should be `.toFixed(2)`.
- **Metadata location depends on event type:** if switching to `payment_intent.succeeded`, `event.data.object` is a `PaymentIntent` and `metadata.orderId` is set directly by your own code when creating the intent. If staying with `charge.succeeded`, confirm in a real test event that the metadata was actually copied onto the

Charge object.

- **No idempotency check:** Stripe retries webhook deliveries on slow responses or errors, so the same event can arrive twice. `updateOrderToPaid` should check if the order is already marked `isPaid` and skip if so, to avoid double-processing.

Suggested signature-verification fix:

```
let event: Stripe.Event;
try {
  event = Stripe.webhooks.constructEvent(
    await req.text(),
    req.headers.get('stripe-signature') as string,
    process.env.STRIPE_WEBHOOK_SECRET as string
  );
} catch (err) {
  return NextResponse.json({ message: 'Invalid signature' }, { status: 400 });
}
```

6. Where the stripe-signature Header Comes From

Stripe attaches a header called `Stripe-Signature` to every webhook POST it sends. It's a cryptographic signature computed from the raw request body combined with your `whsec_` signing secret (the one shown when the event destination was created).

Why it matters: anyone could POST a fake payload to the webhook endpoint pretending to be Stripe (e.g. claiming a payment succeeded, to get an order marked paid for free). The signature proves the request genuinely came from Stripe and wasn't altered in transit.

`constructEvent(body, signature, secret)` recomputes the expected signature from the raw body and `whsec_`, then compares it to the header. A mismatch (wrong secret, tampered body, forged request) causes it to throw.

Important implementation detail: `constructEvent` needs the raw, unparsed body (`await req.text()` is correct). Calling `req.json()` first and re-stringifying would change the exact bytes and break the signature check, since even whitespace differences invalidate it.